



bachelor's thesis

Marcus Janum

Typed assembly for worst-case execution time analysis

Date: June 2026

Advisor: Philippe Bonnet

Contents

1	Introduction	4
1.1	Context	4
1.2	Problem	4
1.3	Approach	4
1.4	Contribution	4
2	ssd_os and the machine	6
3	worst-case execution time analysis	6
4	Typed Assembly Languages	9
4.1	TAL-0	9
4.2	Dependent Types	11
5	Exploring the options	12
5.1	Typing memory	13
6	Designing a language	13
6.1	Strays from risc-v	14
6.2	Types	14
6.3	Collections	16
6.3.1	Stacks	17
6.3.2	Arrays	18
6.3.3	Array allocation using stacks	18
6.4	Abstract machine	19
6.5	Dynamic Semantics	19
6.6	Static Semantics	20
6.7	Type Equality and Coercion	23
6.8	Code Generation	25
7	Implementation	26
7.1	Domain	26
7.2	The tool	26
7.2.1	Flattening	28
7.2.2	Evaluation	30
7.2.3	Coercion	30
7.3	code generation	30
8	Evaluation	31
8.1	Results	31

9	Conclusion	31
9.1	Future work	32

List of Figures

1	Structure of an allocator	6
2	Example of an allocator function	7
3	instructions and operands for the TAL-0 language	9
4	Syntax for a typed assembly language.	15
5	A simple allocation function	17
6	Example code for allocating a stack array and writing to it	18
7	Evaluation rules	20
8	Typing rules for values	21
9	Typing rules	22
10	Type equality rules	23
11	Type coercion rules	24
12	Code generation	25
13	A TAL program	27

1 Introduction

1.1 Context

ssd_os is an operating system for solid state drives (ssd) equipped with RISC-V processors, that provides a synchronous and deterministic programming model. A program is a collection of pipelines; each pipeline is a linear sequence of stages that execute to completion within a tick, i.e., a fixed period of time. Ideally, the programming toolchain verifies that a given stage can execute within a tick. This requires a static analysis of the RISC-V assembly program to reason about the deterministic nature of a stage or its worst-case execution time. In this project, we base our approach on typed assembler program. We rely on the assembly type system to reason about determinism or running time.

1.2 Problem

In this project the problem was reduced to designing a subset of a typed variant of risc-v to model a specific memory structure that used scratch and shared memory regions, and to use this language and abstract machine model to reason about the worst-case execution time of a program written in the typed assembly language.

1.3 Approach

The approach was design heavy in the sense that the language and tool were new. Initially, a lot of time was spent on considering in what way typed assembly (a tool used for correctness) could be used as a tool for execution time analysis.

1.4 Contribution

The contents of this project — what has been done, can be summarized as the following

- A brief presentation of the target system
- A high level overview of methods of worst-case execution time analysis
- An exploration of typed assembly
- How one may use typed assembly for worst-case execution time
- Designing a language based on the previous results and considerations

- and implementing a tool that analyses programs written in said language to reason about execution-time analysis.

```

struct memory_region
{
    uint8_t *start;
    size_t   size;
};

struct arena_allocator
{
    struct memory_region region;
    size_t               allocated;
};

```

Figure 1: Structure of an allocator

2 `ssd_os` and the machine

`ssd_os` is an operating system for solid state drives running on risc-v processors. Programs running on the system is divided into ticks in a pipeline, where each tick is a fixed time interval.

One of the target machines, MicroBlaze-V [1] use — instead of caches for its memory hierarchy — scratch and shared memory. For a multi processor system, this means that each processor has its own scratch memory. Being wired specifically for each processor, this gives faster memory speeds. The other memory type is shared, of which each processor is wired to. This is however slower.

Allocation on this system is done via application specific arena allocators, such as in figure 1 with an accompanying allocation function in figure 2. A simpler allocator simply returns the `start + allocated` pointer and increases the `allocated` field by how much is being requested. This is essentially a stack.

3 worst-case execution time analysis

Worst-case execution time analysis is an important step in the process of developing real-time systems [9] like `ssd_os`. In general, and is easily exemplified, not all programs are subject to execution time analysis, nor do all programs exit. The latter is a consequence of the halting problem [11, ch. 13] [9].

During program execution the execution time is influenced by different factors. For one, the machine; processor frequency, memory speed, memory layout, pipelines. Within a program, the path of execution or control-flow can influence execution time. Depending on the operating systems scheduler guarantees on

```

uint8_t *arena_alloc(struct arena_allocator *alloc, size_t size
, size_t alignment)
{
    uint8_t      *free_region_start = alloc->region.start +
        alloc->allocated;
    const size_t alignment_padding =
        (alignment - (((size_t) free_region_start) %
            alignment)) %
            alignment;
    const size_t size_to_alloc = size + alignment_padding;

    const size_t remaining =
        (alloc->region.start + alloc->region.size) -
            free_region_start;

    if (remaining < size_to_alloc)
        return NULL;

    uint8_t *allocated = free_region_start + alignment_padding;
    alloc->allocated += size_to_alloc;
    return allocated;
}

```

Figure 2: Example of an allocator function

time bounds may or may not be available. One may have timed a userspace program, but what happens when a ps/2 keyboard sends an interrupt? Naturally normal program execution must pause, which may cause the expected time for said program to increase.

Modern hardware is severely limited by memory speed, often called the memory wall [4]. To this end, complex memory layouts and cache hierarchies are used to limit the time it takes to access memory. From this, notions of cache coherence and locality are worked into programming practices to improve performance. Pipelines can start work on other instructions while waiting for memory. However, a program rarely has control over the cache and rarely is a program run in isolation, making it nontrivial to predict memory performance. This means that you can not generally reason about the cache, as even if you know a program just had a cache miss and is going to fetch, another process may do the same, causing an unexpected cache miss again.

Loops and therefore conditional branching are an important part of any computer program, and depending on what path a program may take, the execution time will vary. As mentioned, it is not generally trivial to reason about execution time of programs at compile time, which, besides the architecture such, can be caused simply from the language constructs. Recursion is perhaps the simplest form of looping where halting is not trivial. Some loops *can* be reasoned about at compile time — these are called type-1 loops [11]. Loops of this type range over a fixed set of iterations, such as

```
for (int i = 0; i < 16; i++)  
    do something..
```

The structure of a type-1 for loop is something that can be reasoned about in a high level language, but the compiled programs assembly does not retain this information, since it will be compiled down to conditional and unconditional jumps.

WCET analysis does not necessarily have to be statically determined. Measurements, observed execution time One can use empirical results to get an estimate of execution time of a program by repeatedly running said program and conducting end-to-end measurements. This produces a *maximal observed execution time*, but gives no guarantee of an actual upper bound of execution time and will therefore in general underestimate the worst-case execution time [9]. This (pragmatic) approach is sufficient in a lot of cases and much simpler to execute.

For real-time systems, it needs to be proven that a program will not exceed some time limit, of which measurements are not enough.

The top-down view of static analysis consists in taking (possibly annotated) source code and analyzing the paths through a control-flow graph, and using an

$r ::=$	$\text{r1} \mid \dots \mid \text{rk}$	<i>registers:</i>	$\iota ::=$	$r_d := v$	<i>instructions:</i>
$v ::=$	n	<i>operands:</i>		$r_d := r_s + v$	
	ℓ	<i>integer literal</i>		if r jump v	
	r	<i>label or pointer</i>	$I ::=$	jump v	<i>instruction sequences:</i>
		<i>registers</i>		$\iota; I$	

Figure 3: instructions and operands for the TAL-0 language

abstract machine or other specification of a concrete hardware “sum up” up the code. The source code may be annotated such as in PCC [8] or TAL [5] (which is my focus), or it may be regular code [2] constrained to a subset of the original language, such as C with clear control flow, i.e. bounded loops and no recursion. By using a high-level language, abstractions are introduced that may also aid in determining the intent of code and construct a more precise control-flow graph, eliminating dead branches.

4 Typed Assembly Languages

Typed assembly languages are a group of languages (not unlike assembly) or rather just an idea that in varying degrees introduce type safety to programs written or compiled to assembly. The primary motivation for typed assembly languages is type safety and is strongly related to proof-carrying code [8, ch. 5].

Type checking is an integral part of modern programming languages due to the conveniences they afford. Values may have different properties, such as the differences between boolean and integer values and the operations permitted on them, or as will the topic surrounding the explanation of TAL-0, labels or addresses and regular values. Modern type systems can catch a large range of semantic errors such as jumping to a garbage value or memory safety either at compile time, run time, or a mix of both [11].

4.1 TAL-0

As with PCC, it is of great interest to prove correctness of code. In modern programming languages this is done in part with a strongly typed syntax and semantics as “well-typed programs cannot ‘go wrong’”, thus moving some of the burden of correctness to the type system. The goal of TAL-0 (besides being

a paedological tool from Morrisett to introduce readers to typed assembly) is to restrict control-flow to ensure control-flow safety of assembly code.

An example of a potential mishap — and how it may look solved — may be as the following risc-v snippet; jumping to the contents of a register, *hopefully* an address.

```
jump :
      jr      a0
```

This short example showcases how naive the machine is, as no checks need be performed, and no assurance is made, that ‘a0’ does indeed contain a valid code address. TAL-0 solves this by special syntax and an abstract machine that only allows some evaluation; the above assembly snippet would be disallowed by the evaluation rules. Evaluation rules enable abstract machines to transition, i.e. go from machine M to M' . The above jump would be evaluated with the following evaluation rule

$$\frac{H(\hat{R}(v)) = I}{(H, R, \text{jump } v) \rightarrow (H, R, I)} \text{ (JUMP)}$$

Where H is a heap mapping that maps labels ℓ to heap values $h ::= I$, R is a register file that maps register r_n to the value (or operand, as TAL-0 refers them as) v_n and $\hat{R}$ lifts R to work on operands;

$$\begin{aligned}\hat{R}(r) &= R(r) \\ \hat{R}(n) &= n \\ \hat{R}(\ell) &= \ell\end{aligned}$$

and an abstract machine M is defined as the tuple (H, R, I) and an evaluation evaluates an abstract machine state and returns another state $M \rightarrow M'$. This makes it impossible for a machine to jump to a nonsensical value — however it does not preemptively flag the program as incorrect. In other words, the machine can get stuck, since not every well-formed machine can be evaluated, i.e. there does not exist an evaluation such that $M \rightarrow M'$.

By introducing a type system, one can prevent the abstract machine from reaching a stuck state. To handle the earlier example with jumps, the following rules would prevent the machine from getting stuck.

$$\frac{\Psi; \Gamma \vdash v : \text{code}(\Gamma)}{\Psi \vdash \text{jump } v : \text{code}(\Gamma)} \text{ (S-ADD)}$$

$$\Psi \vdash \ell : \Psi(\ell) \text{ (S-LAB)}$$

where Ψ is a heap that maps labels ℓ to types τ , Γ is a register file type that map registers ρ to types τ , and $\tau ::= \text{code}(\Gamma)$ is a label to some code. The above judgements says that the type of a jump is that of the value iff the value is a valid address.

4.2 Dependent Types

Dependent types introduce singleton types that are constructed with an argument, and whom may interact with each other in the sense that for two values $x: \text{int}(3)$ and $y: \text{int}(4)$, the expression $x + y$ would return a value with the type $\text{int}(7)$. A dependent type may be defined as $\tau ::= \text{int}(x)$ [10] where x is an index expression. Index expressions may be integer literals i , type variables a or arithmetic operations on other index expressions. Index expressions are used within index contexts Φ where a type variable may be bound to an index *sort* (like a type, but separate from the "regular" type system). Within here, index propositions P on index expressions may occur, which are logic operations on index expressions. This is explained in Xi's *A dependently typed assembly language*.

With the introduction of dependent types, it is no longer trivial to determine whether types are equal; we are no longer concerned whether $\text{int} \equiv \text{int}$, but rather $\text{int}(x) \equiv \text{int}(y)$, where x and y both are index expressions that may contain references to type variables and arithmetic operations. To determine the equality one must use constraint module satisfaction [10], a type of SAT solving for first order logic, which definitely is not trivial [3].

Dependent types provide a way to ensure safe array indexing, as one may define a sum function as the following risc-v styled TAL:

```
sum:    {n:nat}[a0:int array(n), a1:int array(n)]
        li      a2, 0
        li      a3, 0

loop:   {n:nat, i:nat, c:int | i < n}
        [a0:int array(n), a1:int array(n), a2: int(i), a3: int(
          c)]
        bge     a2, a1, finish // i >= n
        muli    a5, a2, 4
        add     a5, a5, a0
        lw      a4, 0(a5)
        add     a3, a3, a4      // c+= a0[i]
        addi    a2, a2, 1      // i++
        j       loop

finish: {c:int}[a3: int(c)]
```

mv a0, a3

which is still doing unsafe arithmetic, but if you limited the language to indexing exactly like this, you would eliminate indexing out of bounds. So while dependent types is not immediately sufficient in a risc-v language, it does not require many changes. The code examples indexing could be replaced by a pseudo instruction `load  $r_d$ ,  $v(r_s)$` where value is either an immediate value or a register containing a natural number less than the length of the array in r_s .

An important distinction between type bound registers in TAL and type bound variables in most high-level languages, is that registers may change their type. For example, in the previous code example, the type of `a5` is *top* (no value) until it is assigned in `mul.i`, of where the type — depending on how types are propagated — *int* or *int($i * 4$)*. This also has the consequence that while $i < n$ at the beginning of the block, it is not promised throughout the block execution, and so indexing out of bounds may still happen depending on how types are propagated, and whether they can be determined at compile time. The language in Xis DTAL still inserts runtime checks whenever it can not be statically verified through the type system.

Conceptually one may think of typed assembly as entry conditions for a block.

5 Exploring the options

This section covers the different ways one may use typed assembly languages in aiding with worst-case execution time analysis. Somehow we need to use a tool (TAL) usually attributed to safety and correctness for execution time analysis. At this point we know that we need static analysis, and that we can design a low level language that may be annotated with type information. The type rules may in turn provide safer code, but they could encode promises of values as with dependent types. As mentioned in section 3 the information regarding a loop type is lost in compilation. This is similar to the complaint and motivation behind TAL in that the last 20% [5] or so of compilation is untyped.

One option is to use dependent types to aide the traversal of a control-flow graph of a program and using the index constraints to prove the termination of loops and to reason about the iterations. If simplifying a machine such that all operations take equal time, then the time it takes for each block to execute is a set of values corresponding each path within a block. This means that a block without any jumps would take the same time to execute each time it was executed; a block with one jump would take one of two execution times. For multiple jumps, it either branches or it does not. One could then construct a

graph of blocks, where each edge is a jump to another block, and where each edge is decorated with the branch condition (`bge`, `glt`, etc.) and the types of the registers.

5.1 Typing memory

Another idea is to focus on the second part of static analysis, namely the machine. This is what will be implemented.

As mentioned in section 3 memory timing is difficult to decide because it usually involves layers of cache, and is generally outside the reach of an application programmer. That is, the programmer nor the program decide when and what data is stored in different types of memory. This is not the case in the system described in section 2. There, data needs to be explicitly moved to different memory regions, which boils down to different address ranges. If we assume that an existing TAL has collection types such as array types [10] or stacks [6], we can wrap these in memory regions. Instead of having an *int array(i)* you have a *scratch int array(i)* then this type information can be used to deduce the execution time, given that the memory access time remain constant for the regions. Type variables for memory types can be introduced to make procedures polymorphic on memory types, meaning that code need not be written for *scratch* *and* *shared*.

Relating back to section 3, assembly code would be annotated with type information. The potential memory types is part of the specification of an abstract machine that, paired with machine specific information on how long reads and writes takes for each region can determine an upper bound for the execution time of a program.

The design of the language and the specifics will be explored and explained in section 6.

6 Designing a language

This section describes a language suited for the worst-case execution time analysis tool presented and described in section 5.1. It covers the process and considerations of designing the language, as well as evaluation rules for an abstract machine, type rules, and importantly for the tool, type equality and coercion needed to coerce register files.

“The language” refers to the implementation TAL covered in this section, of which its syntax is presented in figure 4. All registers in a register file map to `top`, except when otherwise noted. The register names follow the risc-v ABI

names [7]. The jump instruction `j` is kept only to link blocks together, mimicking assembly fall-through.

6.1 Strays from risc-v

As with the TALs introduced above, there is little reason to distinguish between immediate instructions and register instructions, meaning that `subi` and `sub` can be treated the same. This is due to the value type that they all share in some form, defined in some way as

$$\text{value } v ::= c \mid r$$

as in figure 4. For code generation, these will have to be reintroduced, but this is trivial, since the value field will either be an immediate value or a register. This is described in section 6.8.

Since all types are word sized, we replace the load and store instructions with `load  $r_d$ ,  $imm(r_s)$` and `store  $r_s$ ,  $imm(r_d)$` where *imm* is a per-word offset, instead of the usual byte offset.

Two very specific instructions are also added: `mvscr` and `mvshr`. They serve a very specific purpose in moving the start address of either the `scratch` or `shared` memory regions into register r_d . Initially I did not want to stray from risc-v, so using a combination of `lui` and `addi`, however that would create an incomplete type between the two instructions as well as having no real way of distinguishing between a number and an address. For a while, the second operand of `mv` could be a value type v , which would then be translated to `lui` and `addi` if the type of v called for it. But still, magic values for the memory region pointers were needed. This is the problem `mvscr` and `mvshr` attempt to solve, however ugly it may be.

6.2 Types

While the syntax of the language is inspired by DTAL, it is much simplified. This is also the case of the τ types, where only *a*, *top*, *int*, and *int(i)* remain. Where in DTAL *int(i)* depended on the type index expressions, I have reduced it to only work on constant values. The array and stack types are still present but wrapped, but collections will be explained later. Universally quantified types, i.e. type variables are still present, although they do not serve a concrete function in terms of the functionality wanted with respects to the tool. Stack type variables are still present, useful for working with `sp`. They are needed to append to a stack, which is needed for `incr`. Specific for the language, region memory type variables have been introduced. They allow labels to be polymorphic over

type variables	α	
State type	σ	$::= \lambda \Delta. R$
Register files	R	$::= [\text{zero} : \text{int}, \text{ra} : \tau_1, \text{sp} : S, \dots]$
region memory types variables	μ	
region memory types	M	$::= \text{scratch} \mid \text{shared} \mid \mu$
stack types variables	ρ	
stack types	S	$::= [] \mid \rho \mid \tau :: S$
Collection types	C	$::= M \tau \text{array}(i) \mid M S$
types	τ	$::= \alpha \mid \text{top} \mid \text{int} \mid \text{int}(i) \mid C$
type variable contexts	Δ	$::= \cdot_{\text{tv}} \mid \Delta, \mu : M \mid \Delta, \rho : S \mid \Delta, \alpha : \tau$
integer literals	i	
constants	c	$::= i \mid l$
values	v	$::= c \mid r$
labels	l	
instructions	ins	$::=$ $\begin{array}{l} aop\ r_d,\ r_s,\ v \\ \mid \text{incr}\ r_d,\ r_s,\ v \\ \mid \text{decr}\ r_d,\ r_s,\ v \\ \mid \text{mv}\ r_d,\ r_s \\ \mid \text{j}\ v \\ \mid \text{halt} \\ \mid \text{mvscr}\ r_d \\ \mid \text{mvshr}\ r_d \\ \mid \text{load}\ r_d,\ i(r_s) \\ \mid \text{store}\ r_s,\ i(r_d) \end{array}$
arithmetic operations	aop	$::= \text{add} \mid \text{sub} \mid \text{mul} \mid \text{div}$
instruction sequences	I	$::= \text{j}\ v \mid \text{halt} \mid ins; I$
Blocks	B	$::= \lambda \Delta. (R, I)$
Label mappings	Λ	$::= \{l_1 : \sigma_1, \dots, l_n : \sigma_n\}$
Programs	P	$::= l_1 : B_1; \dots; l_n : B_n$

Figure 4: Syntax for a typed assembly language.

regions. This is an ergonomic must, as otherwise the programmer would have to write labels for both `scratch` and `shared`.

For the register file, this also means that any type that is not explicitly typed defaults to `top`. This means that for later blocks, `top` can freely be coerced into a new type, and vice versa. That is, any type information can be discarded.

The dependent type $\text{int}(x)$ where x is an index expression has been reintroduced as $\text{int}(i)$ to allow static evaluation of stack types in `incr` and `decr`. The instructions need to add the right amount of types to the head of the stack. It follows that they should only allow immediate values. This is reflected in their evaluation rules and typing rules in figure 7 and figure 8, respectively.

The type variable context strays from the inspiration (DTAL) by essentially a list of bindings from variables to types. This choice also has a big implication on the type equality and coercion rules, as these now produce a side-effect, i.e. they change or add mappings to the context Δ . Another implication of this is that it forces the entrypoint to have no type variables. This makes sense, as there would be no way to deduce the type information. If for some reason one wanted to explicitly state “anything” one would have to use `top`. If $\alpha \in \text{dom}(\Delta)$ then $\Delta\alpha = \tau$, and similarly for μ and ρ .

6.3 Collections

This section describes how the array and stack types work with each other, and how they are to be thought of conceptually in the language moving forwards. Some TALs introduce `malloc` or similar pseudo-instructions to introduce a notion of types to arrays. Since assembly does not in of itself contain arrays, but only pointers, some level of abstraction needs to be introduced for a type system to be able to type arrays and differentiate them from garbage and labels. In C, heap arrays are pointers, and pointer arithmetic is allowed. Labels can only be used in `goto` statements, and not even arithmetic is permitted on them. Still, it is not trivial whether `float* n` points to a scalar, or is an array.

A `malloc` instruction is not suited in this case, since the target system contains multiple allocators (as discussed in section 2) working over memory regions. A `malloc` function is used in most TALs if they use dynamic memory allocation, because the introduction of types means that there need be a way to distinguish between arithmetic operations and pointer arithmetic. The problem then evolves into the question of how to design a type system that makes an allocator implicitly emit an array type, or a type that can be coerced into arrays. A simple allocator is shown in figure 5. It has no safety checks, and it does not align allocated memory. The TAL type systems often only accept word sized types, and for simplicity, so will I. To answer this, we first look into stacks, arrays, and a


```

unsigned int *simple_malloc(struct arena_allocator *alloc,
    size_t size)
{
    unsigned int* start = alloc->region.start + alloc->
        allocated;
    alloc->allocated += size;
    return start;
}

```

Figure 5: A simple allocation function

type system. Another issue is the fact that the stack pointer grows downwards, while the arena stacks grow upwards. This will also be solved.

6.3.1 Stacks

Stack types can be defined recursively as

$$\begin{array}{ll}
 \text{stack types variables} & \rho \\
 \text{stack types} & S ::= [] \mid \rho \mid \tau :: S
 \end{array}$$

[10]

In DTAL, the stack type is a part of the type state Σ , and is modified through `pushd` and `pop` operations, a very conventional way to use the stack. Here, stacks are used not only for the call-stack, but general allocations, and to be coerced into array types. To communicate to the type system that we intend to modify a stack, we introduce two instructions `incr` and `decr`. They may be described through the code generation in 12. `incr` decreases the stack pointer in r_s by v words (removing the need for runtime alignment) and places the value in r_d whose types will be that of r_s with v top values appended to it, e.g.,

```

main:    [sp: 'r]
        incr    a0, sp, 4    ; a0 : top :: top :: top :: top ::
        'r

```

where `'r` is a stack type variable. `decr` functions as expected, popping v words off the stack. It is important to remember that the call stack grows *downwards*, so `incr` may at first sound counter intuitive because it *decreases* the pointer.

A programmer may not be interested in passing on *the whole* stack after increasing the word count, such as stack arrays in C. If the type of a stack allocated array was the whole call-stack, not only would it obfuscate the intend, it would limit memory safety (if the type system enforced such things) since

```

start:  [a0: int, a1: int, a2: int, a3: int]
alloc:  [a0: int, a1: int, a2: int, a3: int]
        incr    a4, sp, 4          ; a4: top :: top :: top :: top
        :: ..
store:  [a0: int, a1: int, a2: int, a3: int, a4: 'm int array
        (4)]
        store    a0, 0(a4)
        store    a1, 1(a4)
        store    a2, 2(a4)
        store    a3, 3(a4)

```

Figure 6: Example code for allocating a stack array and writing to it

any stack-allocated array would be permitted to access the whole stack. It is therefore desirable to *cut down* or slice a stack. This can be done by considering a stack S a subtype of a stack S' if S is contained within S' with a possibly empty tail. Or formally

$$\tau_0 :: \dots :: \tau_n \leq \tau_0 :: \dots :: \tau_m \mid n \leq m$$

6.3.2 Arrays

Since we have foregone array initialization instructions such as *newarray*, we add the choice to coerce a stack containing only one type to an array of that type. In section 2 it is mentioned that the arena allocators work like a stack.

6.3.3 Array allocation using stacks

At this point we can subtype stacks to create smaller stacks, and we can instantiate array types by coercing a stack.

While the call-stack grows downwards and *incr* and *decr* can be used as expected, the memory region allocations grows upwards. The solution lies in the conceptualization.

While *incr* and *decr* only work on immediate values here, this is only for simplification, and since the language contains no looping constructs, there is no value in having dependent sizes.

An example program that allocates a stack array and stores four integers could look like the program in figure 6.

Another use is working with the arena allocators. They are simple, and also work like a stack, but these grow upwards, which is opposite the call stack and therefore the typing rules would not work.

```

struct memory_region
{
    uint8_t *start;
    size_t   size;
};

struct arena_allocator
{
    struct memory_region region;
    size_t             allocated;
};

```

6.4 Abstract machine

Unlike other TAL abstract machines, I am omitting the stack, since it will be used as a normal register (sp). The DTAL abstract machine has a machine state $\mathcal{M}(\mathcal{H}, \mathcal{R}, \mathcal{S})$ being a triple over a heap, register file, and stack. Since the system the language is targeted at is stack based, no heap is needed (except for program text, but this is omitted partially due to program scope) and no explicit stack is needed, since the register sp will be used. As such, the abstract machine contains only a finite mapping between registers and values.

This is also the motivation behind removing the DTAL type state Σ and state type σ .

6.5 Dynamic Semantics

Similar to the treatment in DTAL, we introduce an instruction sequence J_P of a program P , and I_B for the instruction sequence in B as

$$J_P = l_1; I_{B_1}; \dots; l_n; I_{B_n}$$

We use $J_P(i)$ to refer to the i th instruction or label in J_P . If it is clear what P is being referred to, we may omit the subscript. We also use $J_P^{-1}(l) = i \iff J_P(i) = l$ from DTAL.

A P -snapshot [10] is either `halt` or a tuple $(ic, \mathcal{M})$ such that $0 \leq ic < \text{length}(J_P)$, where ic is the instruction counter. $(ic, \mathcal{M}) \rightarrow_P (ic', \mathcal{M}')$ means that the machine state $\mathcal{M}$ transforms into $\mathcal{M}'$ by executing the instruction at $J_P(ic)$. Similarly ic' is incremented or replaced in the case of `j`.

$$\begin{array}{c}
\frac{J(ic) = \text{add } r_d, r_s, v \quad \mathcal{M}(r_s) = i \quad \mathcal{M}(v) = j}{(ic, \mathcal{R}) \rightarrow_P (ic + 1, \mathcal{R}[r_d = i + j])} \text{ (eval-add)} \\
\frac{J(ic) = \text{incr } r_d, r_s, v \quad \mathcal{M}(r_s) = \mu S \quad \mathcal{M}(v) = i \quad i > 0}{(ic, \mathcal{R}) \rightarrow_P (ic + 1, \mathcal{R}[r_d = \mu \text{top}_0 :: \dots :: \text{top}_i :: S])} \text{ (eval-incr)} \\
\frac{J(ic) = \text{incr } r_d, r_s, v \quad \mathcal{M}(r_s) = \mu \text{top}_0 :: \dots :: \text{top}_i :: S \quad \mathcal{M}(v) = i \quad i > 0}{(ic, \mathcal{R}) \rightarrow_P (ic + 1, \mathcal{R}[r_d = \mu S])} \text{ (eval-decr)} \\
\frac{J(ic) = \text{mvscr } r_d}{(ic, \mathcal{R}) \rightarrow_P (ic + 1, \mathcal{R}[r_d = \text{scratch } \rho_{\text{scratch}}])} \text{ (eval-mvscratch)}
\end{array}$$

Figure 7: Evaluation rules

$$\mathcal{M}(v) = \begin{cases} i & \text{if } v \text{ is integer } i \\ l & \text{if } v \text{ is label } l \\ C & \text{if } v \text{ is collection } C \\ \mathcal{R}(r) & \text{if } v \text{ is register } r \end{cases}$$

Similar to DTAL.

The evaluation of most instructions is given in figure 7, while the rest is easily inferred. For eval-mvscratch, ρ_{scratch} refers to the stack value $\text{top}_1 :: \dots :: \text{top}_n$ where n is the size of the memory region in words.

6.6 Static Semantics

The typing rules for the language is the most important step in the solution of calculating worst-case execution time with the method discussed.

One difference between other TALs and especially how values v in DTAL and how they behave here, is that we need a way to instantiate the region memory types. Earlier instantiating was presented as

In figure 9 presenting the typing rules, ρ_{scratch} and ρ_{shared} represent two stack types. Each region should be defaulted to top , and the amount depends on the physical size. Since the scratchpad is 128KB [1], ρ_{scratch} may be defined as $\text{top}_1 :: \dots :: \text{top}_{128/w \cdot 1000} :: []$ where w is the machines wordsize.

The type rules for values in figure 8 depend on the label mapping Λ for typing labels. This is the meaning of the notation $\vdash_{\Lambda}$, where Λ is a label mapping function. In practice, only one label mapping function will be used, so for brevity $\vdash$ may be used — like in DTAL.

The typing rules for arithmetic instructions follow from add in figure 9.

The typing for jumps j do not continue checking the instructions of the target block. It checks if the current state aligns with the state type σ in the target block.

$$\begin{array}{c}
\frac{}{\Delta; R \vdash_{\Delta} i : \text{int}(i)} \text{ (type-immediate-int)} \\
\frac{\Delta(l) = \sigma}{\Delta; R \vdash_{\Delta} l : \sigma} \text{ (type-label)} \\
\frac{}{\Delta; R \vdash_{\Delta} r : R(r)} \text{ (type-reg)} \\
\frac{\Delta(\alpha) = \tau}{\Delta; R \vdash_{\Delta} \alpha : \tau} \text{ (type-var)} \\
\frac{\Delta(\rho) = S}{\Delta; R \vdash_{\Delta} \rho : S} \text{ (type-stack-var)} \\
\frac{\Delta(\mu) = M}{\Delta; R \vdash_{\Delta} \mu : M} \text{ (type-memory-var)}
\end{array}$$

Figure 8: Typing rules for values

This means comparing each register type. This is explained in section 6.7. Since the handling of stacks and arrays rely heavily on subtyping, we use the coercion rules for comparison or acceptance of a new state type.

The type rules differ from DTAL in that they use and (potentially) *update* the type variable context Δ . This is specifically for variable typing rules. The motivation for this choice is jumps, going from one machine state to another. To make use of the ergonomics discussed earlier in regards to handling stacks and arrays, a machine state M consisting of a register file and variable mapping can jump to M' iff M can be coerced into M' . Or in other words

$$\Delta, R \models_c R' \rightsquigarrow \Delta'$$

When adding a binding to a type variable context, we do not want to accidentally define circular types such as $\alpha : \beta, \beta : \alpha$. This is why the variable type equality and coercion do not have rules for other variables, and as a consequence, type variable contexts can only have a depth of one, if one were to imagine Δ as a graph. This is not dissimilar in the treatment of existential qualifiers [3], in the sense of a graph.

Because of this constraint type variables cannot refer to other type variables, when we coerce register files we can assert that at type-coerce-regfile, type-reg will never evaluate to a type variable. This means that there is no need to check if variables in the two register files are unique, or in other words, we do not have to create fresh variables, something that is required for hindley-milner type inference [11].

$$\begin{array}{c}
\frac{\Delta; R \vdash r_s : \text{int} \quad \Delta; R \vdash v : \text{int} \quad \Delta; R \vdash I}{\Delta; R \vdash \text{add } r_d, r_s, v; I} \text{ (type-add)} \\
\frac{\Delta; R \vdash r_s : \mu S \quad \Delta; R \vdash v : \text{int}(i) \quad i > 0 \quad \Delta; R[r_d = \mu \text{top}_1 :: \dots :: \text{top}_i :: S] \vdash I}{\Delta; R \vdash \text{incr } r_d, r_s, v; I} \text{ (type-incr)} \\
\frac{\Delta; R \vdash r_s : \mu \tau_1 :: \dots :: \tau_i :: S \quad \Delta; R \vdash v : \text{int}(i) \quad i > 0 \quad \Delta; R[r_d = \mu S] \vdash I}{\Delta; R \vdash \text{decr } r_d, r_s, v; I} \text{ (type-decr)} \\
\frac{\Delta; R \vdash r_s : \tau \quad \Delta; R[r_d = \tau] \vdash I}{\Delta; R \vdash \text{mv } r_d, r_s; I} \text{ (type-mv)} \\
\frac{\Delta; R[r_d = \text{scratch } \rho_{\text{scratch}}] \vdash I}{\Delta; R \vdash \text{mvscr } r_d; I} \text{ (type-mvscratch)} \\
\frac{\Delta; R[r_d = \text{shared } \rho_{\text{shared}}] \vdash I}{\Delta; R \vdash \text{mvshr } r_d; I} \text{ (type-mvshared)} \\
\frac{\Delta; R \vdash v : \lambda \cdot_{\text{tv}} . R' \quad \Delta; R \models_c R' \rightsquigarrow \Delta'}{\Delta; R \vdash \text{j } v} \text{ (type-jump)} \\
\frac{\Delta; R \vdash v : \text{int}(i) \quad \Delta; R \vdash r_s : M\tau_{\text{array}}(j) \quad i < j \quad \Delta; R[r_d = \tau] \vdash I}{\Delta; R \vdash \text{load } r_d, v(r_s); I} \text{ (type-load-array)} \\
\frac{\Delta; R \vdash v : \text{int}(i) \quad i \geq 0 \quad \Delta; R \vdash r_s : M\tau_0 :: \dots :: \tau_i :: S \quad \Delta; R[r_d = \tau_i] \vdash I}{\Delta; R \vdash \text{load } r_d, v(r_s); I} \text{ (type-load-stack)} \\
\frac{\Delta; R \vdash v : \text{int}(i) \quad \Delta; R \vdash r_s : \tau \quad \Delta; R \vdash r_d : M\tau'_{\text{array}}(j) \quad i < j \quad \Delta \models \tau' \leq \tau \quad \Delta; R \vdash I}{\Delta; R \vdash \text{store } r_s, v(r_d); I} \text{ (type-store-array)} \\
\frac{\Delta; R \vdash r_s : \tau \quad \Delta \models \tau' \leq \tau \quad \Delta; R[r_d = M\tau_0 :: \dots :: \tau_i :: S] \vdash I \quad \Delta; R \vdash v : \text{int}(i) \quad i \geq 0 \quad \Delta; R \vdash r_d : M\tau_0 :: \dots :: \tau_i :: S}{\Delta; R \vdash \text{store } r_s, v(r_d); I} \text{ (type-store-stack)}
\end{array}$$

Figure 9: Typing rules

$$\begin{array}{c}
\frac{\alpha \in \Delta}{\Delta \models \alpha \equiv \alpha} \text{ (type-eq-var)} \\
\frac{}{\Delta \models \text{top} \equiv \text{top}} \text{ (type-eq-top)} \\
\frac{}{\Delta \models \text{int} \equiv \text{int}} \text{ (type-eq-int)} \\
\frac{\Delta \models \tau \equiv \tau' \quad \Delta \models S \equiv S'}{\Delta \models \tau :: S \equiv \tau' :: S'} \text{ (type-eq-stack)} \\
\frac{}{\Delta \models [] \equiv []} \text{ (type-eq-emptystack)} \\
\frac{\Delta \models \tau \equiv \tau' \quad i = j}{\Delta \models \tau \text{ array}(i) \equiv \tau' \text{ array}(j)} \text{ (type-eq-array)} \\
\frac{}{\Delta \models \text{scratch} \equiv \text{scratch}} \text{ (region-eq-scratch)} \\
\frac{}{\Delta \models \text{shared} \equiv \text{shared}} \text{ (region-eq-shared)} \\
\frac{\Delta \models \mu \equiv \mu' \quad \Delta \models c \equiv c'}{\Delta \models \mu c \equiv \mu' c'} \text{ (type-eq-collection)}
\end{array}$$

Figure 10: Type equality rules

6.7 Type Equality and Coercion

We start by introducing the type equality and coercion rules.

The typing rules for collection types contain a lowercase c , which is taken to mean the pattern matched type of either $\tau \text{ array}(i)$ or S .

The coercion rules allow the type variable context to be modified, by coercing type variables into types. The use case is limited to state transitions, i.e. jumps. This is the part that binds type variables to types, specifically

$$\frac{\Delta; R \vdash r_n : \tau_n \quad \Delta_{n-1}; R' \vdash r_n : \tau'_n \quad \cdot_{\text{tv}} \models \tau'_n \leq \tau_n \rightsquigarrow \Delta_n}{\Delta; R \models_c R' \rightsquigarrow \Delta_n}$$

where Δ_n is created through accumulating type captures starting at $\Delta_1 = \cdot_{\text{tv}}$ for all n registers.

Since I decided to reflect the risc-v register ABI names in the syntax of the language, it is not actually possible to iterate over all registers. Not for any semantic reasons. Therefore I informally introduce r_n to be a register, and the above meaning is that it iterates over all (n) registers.

The coercion rules make it possible to capture type variables and assign them a type. The “instantiation” rules refer to those where the type variable is not already in the variable context; the binding is inserted into the context. It is quite conceivable that the same type variable be used multiple times in the same register file, in this case the coercion type needs to be equal to the type already bound to the type variable in the type variable context.

$$\begin{array}{c}
\frac{\alpha \in \mathbf{dom}(\Delta)}{\Delta \models \alpha \leq \alpha \rightsquigarrow \Delta} \text{ (type-coerce-var)} \\
\frac{\alpha \notin \mathbf{dom}(\Delta)}{\Delta \models \alpha \leq \text{int} \rightsquigarrow \Delta[\alpha: \text{int}]} \text{ (type-coerce-var-int)} \\
\frac{\alpha \notin \mathbf{dom}(\Delta)}{\Delta \models \alpha \leq \text{int}(i) \rightsquigarrow \Delta[\alpha: \text{int}(i)]} \text{ (type-coerce-var-imm)} \\
\frac{\alpha \notin \mathbf{dom}(\Delta)}{\Delta \models \alpha \leq C \rightsquigarrow \Delta[\alpha: C]} \text{ (type-coerce-var-collection)} \\
\frac{\Delta(\alpha) = \tau' \quad \Delta \models \tau \equiv \tau'}{\Delta \models \alpha \leq \tau \rightsquigarrow \Delta} \text{ (type-coerce-var-t)} \\
\frac{\rho \notin \mathbf{dom}(\Delta)}{\Delta \models \rho \leq \rho \rightsquigarrow \Delta} \text{ (type-coerce-stack-var)} \\
\frac{\rho \notin \mathbf{dom}(\Delta)}{\Delta \models \rho \leq [] \rightsquigarrow \Delta[\rho: []]} \text{ (type-coerce-stack-var-empty)} \\
\frac{\rho \notin \mathbf{dom}(\Delta) \quad \cdot_{\text{tv}} \models \alpha \leq \Delta \models \rho \leq S \rightsquigarrow \Delta'}{\Delta \models \rho \leq \tau :: S \rightsquigarrow \Delta[\rho: \tau :: \Delta'(\rho)]} \text{ (type-coerce-stack-var-cons)} \\
\frac{\mu \in \mathbf{dom}(\Delta)}{\Delta \models \mu \leq \mu \rightsquigarrow \Delta} \text{ (type-coerce-memory-var)} \\
\frac{\mu \notin \mathbf{dom}(\Delta)}{\Delta \models \mu \leq \text{shared} \rightsquigarrow \Delta[\mu: \text{shared}]} \text{ (type-coerce-memory-var-shared)} \\
\frac{\mu \notin \mathbf{dom}(\Delta)}{\Delta \models \mu \leq \text{scratch} \rightsquigarrow \Delta[\mu: \text{scratch}]} \text{ type-coerce-memory-var-scratch} \\
\frac{\tau \notin \mathbf{dom}(\Delta)}{\Delta \models \tau \leq \text{top} \rightsquigarrow \Delta} \text{ (type-coerce-top)} \\
\frac{}{\Delta \models \text{int} \leq \text{int} \rightsquigarrow \Delta} \text{ (type-coerce-int)} \\
\frac{\Delta \models s \leq s' \rightsquigarrow \Delta' \quad \Delta' \models \tau \leq \tau' \rightsquigarrow \Delta''}{\Delta \models \tau :: S \leq \tau' :: S' \rightsquigarrow \Delta''} \text{ (type-coerce-stack)} \\
\frac{\Delta \models \tau \leq \tau' \rightsquigarrow \Delta' \quad i \leq j}{\Delta \models \tau \text{ array}(i) \leq \tau' \text{ array}(j) \rightsquigarrow \Delta'} \text{ (type-coerce-array)} \\
\frac{\Delta \models \tau \equiv \tau_0 \quad \dots \quad \Delta \models \tau \equiv \tau_i}{\Delta \models \tau \text{ array}(i) \leq \tau_0 :: \dots :: \tau_i :: S} \text{ (type-coerce-array-stack)} \\
\frac{\Delta \models \mu \leq \mu' \rightsquigarrow \Delta' \quad \Delta' \models c \leq c' \rightsquigarrow \Delta''}{\Delta \models \mu c \leq \mu' c' \rightsquigarrow \Delta''} \text{ (type-coerce-collection)} \\
\frac{\Delta; R \vdash r_n : \tau_n \quad \Delta_{n-1}; R' \vdash r_n : \tau'_n \quad \Delta_{n-1} \models \tau'_n \leq \tau_n \rightsquigarrow \Delta_n}{\Delta; R \models_c R' \rightsquigarrow \Delta_n} \text{ (type-coerce-regfile)}
\end{array}$$

Figure 11: Type coercion rules

$$\begin{aligned}
\text{add } r_d, r_s, v &\mapsto \begin{cases} \text{add} & r_d, r_s, r \text{ iff } v \text{ is register } r \\ \text{addi} & r_d, r_s, i \text{ iff } \Delta; R \vdash_{\Lambda} v : \text{int}(i) \end{cases} \\
\text{load } r_d, v(r_s) &\mapsto \begin{cases} \text{lw} & r_d, (i \cdot \text{wordsize})(r_s) \text{ iff } \Delta; R \vdash_{\Lambda} v : \text{int}(i) \end{cases} \\
\text{store } r_s, v(r_d) &\mapsto \begin{cases} \text{sw} & r_s, (i \cdot \text{wordsize})(r_d) \text{ iff } \Delta; R \vdash_{\Lambda} v : \text{int}(i) \end{cases} \\
\text{mvscr } r_s &\mapsto \begin{cases} \text{lui} & r_d, \%hi(\text{SCRATCH}) \\ \text{addi} & r_d, r_d, \%lo(\text{SCRATCH}) \end{cases} \\
\text{mvshr } r_s &\mapsto \begin{cases} \text{lui} & r_d, \%hi(\text{SHARED}) \\ \text{addi} & r_d, r_d, \%lo(\text{SHARED}) \end{cases} \\
\text{incr } r_d, r_s, v &\mapsto \begin{cases} \text{mul} & r_d, r, \text{wordsize} \text{ iff } v \text{ is register } r \\ \text{sub} & r_d, r_s, r_d \\ \text{addi} & r_d, r_s, -i \cdot \text{wordsize} \text{ iff } \Delta; R \vdash_{\Lambda} v : \text{int}(i) \end{cases} \\
\text{decr } r_d, r_s, v &\mapsto \begin{cases} \text{mul} & r_d, r, \text{wordsize} \text{ iff } v \text{ is register } r \\ \text{add} & r_d, r_s, r_d \\ \text{addi} & r_d, r_s, i \text{ iff } \Delta; R \vdash_{\Lambda} v : \text{int}(i) \end{cases}
\end{aligned}$$

Figure 12: Code generation

6.8 Code Generation

The target for code generation is risc-v32. This means that instructions that load addresses such as `mvscr` gets converted to the common `lui-addi` pattern. The reason `wordsize` is not set to 32 is because I wanted to be able to target risc-v64. The changes to code generation is small, since only the address loading has to change, as well as the value `wordsize`. Code generation is in general pretty straight forward, showcased in figure 12. Arithmetic operations expand to either their I or R instruction formats [7].

Code generation contains variables that are set to special values for scratch and shared addresses for `mvscr` and `mvshr`, as well as the `wordsize` — which will be 8. The meta data field in the implementation will be in bits which means that

it gets divided by 8 to get the size in bytes. `mvscr` and `mvshr` were introduced to instantiate the two collection types.

The pseudo instructions could be expanded to handle more value types, such as with `load` and `store`, which only provide code given that v is an immediate value. It is definitely possible to add code in the case where v is a register by using pointer arithmetic. By abstracting away pointer arithmetic, a language can provide better memory safety. The reason pointer arithmetic is problematic, is because a type system can not easily distinguish between addresses and integers. This follows from the reasoning behind the introduction of `mvscr` and `mvshr`. That being said, I did not do this, since memory safety is not my focus.

7 Implementation

The full code will be available in my git repository.

To give the tools the information they need regarding timings and addresses, each program must start with a `.meta` section as in figure 13. The ordering of the fields is absolute, but this is simply a parser issue.

7.1 Domain

Only a small subset of what has been discussed will be implemented, namely typing memory region types for the scratchpad system model. The typing rules will ensure that it is known before runtime of which memory region each load/store operation writes or reads from. Since the system works without cache (scratch region effectively taking its place) all IO has a set cost, and as do the arithmetic instructions. The problem then boils down to essentially summing up the cost of each instruction.

The solution is somewhat novel, since there is a painful lack of control-flow. In the theory description some outline of how this may be thought of and implemented is discussed. The biggest constraint and factor of deciding that control-flow is out of scope is due to the solving process (predicate logic SAT solving).

7.2 The tool

The tool shall take a program in the language specified in section 6, and produce an estimate of its runtime. From the language source code it creates a meta program, that is, a program as described in figure 4 wrapped in some meta data that specifies memory addresses and timings for arithmetic, scratch memory, and shared memory operations. It shall trace the (linear) control-flow

```

.meta
    .wordsize 32
    .scaddr 0x88000000
    .scsize 128000
    .sctime 4
    .shaddr 0x00100000
    .shsize 16000
    .shtime 100
    .artime 1

_start: []
    mvscr    a5
    add      a1, zero, 1
    add      a2, zero, 2
    add      a3, zero, 3
    add      a4, zero, 4
    j        alloc

alloc: [a5: mu rho]
    mv       a0, a5
    decr     a5, a5, 4
    j        write

write:
    [a0 : mu int array(4), a1: int, a2: int, a3: int, a4:
      int]
    store a1, 0(a0)
    store a2, 1(a0)
    store a3, 2(a0)
    store a4, 3(a0)
    load a1, 0(a0)
    load a2, 1(a0)
    load a3, 2(a0)
    load a4, 3(a0)
    halt

```

Figure 13: A TAL program

through the program and decorate each block with type variable contexts and update register files by coercion. This results in a list of instruction, registerfile instruction pair within each block, where each instruction cost can be summed up.

I chose to implement the tool and code generation in haskell because of its pattern matching and because I wanted to try the language. This results in code that may not be haskell-idiomatic, what ever that may entail. I have deliberately chosen not to do complete pattern matches where they are not explicitly necessary because this made it easier to catch missing matches.

Most of the logic can be boiled down to the following function:

```
tool (info, prog) =
  do
    blocks <- trace prog "_start"
    -- append Halt hack such that the last block is
    -- calculated (is first in zip tuple)
    let pairs = zip blocks (tail blocks ++ [Block (\r ->
      Top, [Halt])])
    let delta = TypeVarContext [] [] []
    totalcost delta pairs
  where
    totalcost :: TypeVarContext -> [(Block, Block)] ->
      Maybe Integer
    totalcost _ [] = Just 0
    totalcost delta ((b, Block (rf, _)) : pairs) =
      do
        fb <- flatten delta b          -- flatten
        -- current block.
        eb <- evalB info delta fb      -- eval block
        -- to get block'.
        rf' <- finalrf eb              -- gets the
        -- final regfile.
        delta' <- coerce delta rf rf'  -- coerce next
        -- regfile to resolve type variables
        (+) <$> cost delta eb <*> totalcost delta'
        pairs
```

where totalcost does the heavy lifting. It seems natural to explain the code in the order of execution.

7.2.1 Flattening

flatten replaces occurrences of variables with their bindings in a type variable context. Instances are defined for types, collection types, memory types, and

blocks, which possibly call each other recursively. Flattening a collection type means flattening both the memory type and either a stack type or a type of an array. The simple case is the regular type τ .

```
instance Flatten Type where
  flatten delta (TVar a)           = a 'lookupTD' delta
  flatten delta (TCollection c)    = TCollection <$> flatten
    delta c
  flatten delta t                  = Just t
```

When calculating execution time blocks are flattened as the above code showcases in section 7.2.

```
instance Flatten Block where
  flatten delta (Block (rf, insns)) =
    do
      pairs <- mapM moveM $ zip regs (flatrf <$> regs)
      let rf' = fromMaybe Top . ('lookup' pairs)
      return $ Block (rf', insns)
    where
      regs = [Zero ..]
      flatrf = flatten delta . rf
      moveM :: Monad m => (a, m b) -> m (a, b)
      moveM (a, mb) =
        do
          b <- mb
          return (a, b)
```

To flatten a block, the register file is flattened by flattening the type of each register and returning the resulting register file and the same instruction sequence. A register file is implemented as a function from registers to types.

```
type RegFile = (Register -> Type)
```

Flattening returns a Maybe value of whatever is being flattened. To get back to a register file (which is defined as a function) a function can be defined using the lookup prelude function with a default top. So far, we only have (maybe) the flattened types. To use the lookup function, we create a tuple, but this has the signature (Register, Maybe Type), thus moveM that extends the monad inside the tuple to contain the tuple. Then, because we have a list of (maybe) pairs, mapM turns the monad inside out to get a maybe list. Now the new register file function can be created.

7.2.2 Evaluation

evalB evaluates a block in the sense that it records the register file state for each instruction in a block. The type of this is `Block'`, which is like a `Block`, but is a regfile instruction tuple. The function evaluates an instruction, saves the new regfile and instruction and evaluates the remaining instructions of a block with the new regfile

```
evalB :: Info -> TypeVarContext -> Block -> Maybe Block'
evalB info delta (Block(rf, insn : insns)) =
  do
    rf' <- evalI info delta rf insn
    ((rf', insn) :) <$> evalB info delta (Block (rf', insns
    ))
evalB _ _ (Block (rf, [])) = return []
```

resulting in a *(regfile, instruction)* tuple; a `Block'`. The execution time of a block can then be calculated by folding the result on addition over the cost of each regfile-instruction pair.

Back in `totalcost`, the final regfile is used to coerce into the next blocks register file.

7.2.3 Coercion

```
instance TypeEq Type where
  equiv delta (TVar a) (TVar a') =
    let indom = isJust $ a 'lookupTD' delta
    in indom && a == a'
  equiv delta Top Top = True
  equiv delta TInt TInt = True
  equiv delta (TInt i) (TInt j) = i == j
  equiv delta (TCollection c) (TCollection c') =
    equiv delta c c'
```

7.3 code generation

A separate tool constructs compilable risc-v assembly code based on the code generation presented in section 6.8. It uses the same lexer and parser resulting in the same abstract syntax tree, and then simply matches against figure 12. Based on the meta data provided in the TAL file, it defines the scratch and shared memory regions as preprocessor macros.

8 Evaluation

8.1 Results

Due to an unfortunate lack of availability of the target machine, I was unable to run example programs on the right architecture. However I did have access to another risc-v machine, though with a cached memory layout.

I had wrongfully assumed that the 64bit version of risc-v was compatible with the 32 bit version, which was not entirely correct, since loading a 32 bit address into a register using `lui` and `addi` (as `mvscr` does) does not work, since the address gets sign extended. Loading a 64 bit address will obviously not work with the previous instructions either, since the two immediate fields does not add up to 64 bits. Due to this, I manually changed (as in after code generation) the two instructions that make up `mvscr` to `li` and `slli`.

The programs written in the TAL language are meant to contain "the whole" program it starts at the `_start` label. However, to run it on `ssd_os`, it has to be called from within another program, so obviously the entry point label had to be changed. This is also something I did after code generation. Similarly, the `halt` instruction should not exit the process, so the code generation was changed to the `ret` instruction.

The program in figure 13 was run repeatedly. The first run gave values of 107 and 60 milliseconds, while all repetitions gave exactly 38. This can be attributed to caching. When evaluating the same program with the tool we get the following

```
$ cat ex2.tal | ./tool
48
```

If we imagine that the scratch region is similar to the cached program state, and shared the non-cached, then we can somewhat reason about the correctness of the tool. If we do that, then 48 looks promising. Of course, the meta data can be tuned to get a closer upper bound. If we instead use the shared region by replacing `mvscr` with `mvshr` it takes proportionally longer. The timing in figure 13 for the shared memory is quite large, so we overshoot the non-cached time of the test by a lot. If we set it to 8 instead, we get a timing of 80, which is more reasonable.

9 Conclusion

In this project I have used a method generally used in relation to correctness (typed assembly) as a tool for worst-case execution time. It is only made possible

due to the specific hardware it is being run on that made it possible to type its memory state. The analysis tool only targets a subset of what is required for a real analysis tool, in that it does not support control flow. The tool in of itself is a lesser part of the project. The biggest part was figuring out a language that made the analysis possible. The language is for all intents and purposes not practical, but this was never the goal. It shows that yes indeed TAL can be used for WCET analysis, albeit after jumping through some hoops.

9.1 Future work

The language lacks control-flow, and this would be an obvious successor to this project. It would most likely reimplement some of the features of DTAL and use SAT solving of predicate logic to determine loop invariants as outlined in section 5.

References

- [1] Philippe Bonnet. σ : An operating system for solid-state drives.
- [2] Gerard J. Holzmann. The power of 10: Rules for developing safety-critical code. *Computer*, 39(6):95–97, June 2006.
- [3] Michael Huth and Mark Ryan. *Logic in Computer Science: Modelling and Reasoning About Systems*. Cambridge University Press, 2004.
- [4] Per Stenström Michel Dubois, Murali Annavaram. Parallel computer organization and design.
- [5] Greg Morrisett. From system f to typed assembly language.
- [6] Greg Morrisett. stack-based typed assembly language.
- [7] David A. Patterson and John L. Hennessy. Computer organization and design risc-v edition.
- [8] Benjamin C. Price. Advanced topics in types and programming languages.
- [9] Reinhard Wilhelm, Jakob Engblom, Andreas Ermedahl, Niklas Holsti, Stephan Thesing, David Whalley, Guillem Bernat, Christian Ferdinand, Reinhold Heckmann, Tulika Mitra, Frank Mueller, Isabelle Puaut, Peter Puschner, Jan Staschulat, and Per Stenström. The worst-case execution-time problem—overview of methods and survey of tools. *ACM Trans. Embed. Comput. Syst.*, 7(3), May 2008.
- [10] Hongwei Xi and Robert Harper. A dependently typed assembly language. *SIGPLAN Not.*, 36(10):169–180, October 2001.
- [11] Torben Ægidius Mogensen. Programming language design and implementation.